Sistemi Informativi Evoluti e Big Data – A.A. 2025-2026

Homework #4 – MapReduce

1. Obiettivo e Generazione dei Dati

L'obiettivo dell'homework è progettare un sistema MapReduce per analizzare i transiti autostradali, calcolando il pedaggio medio per tipo di veicolo nel 2025 e la variazione rispetto al decennio precedente (2015).

Per l'esperimento è stato generato un dataset sintetico (caselli autostradali.csv) di 100.000 record tramite uno script Python personalizzato basato sulla libreria Faker.

Nota sulla generazione: Per simulare uno scenario realistico, è stato introdotto un fattore di inflazione variabile (tra il 15% e il 25%) sui prezzi generati per l'anno 2025 rispetto ai prezzi base del 2015.

2. Analisi Quantitativa dei Risultati

L'elaborazione è stata effettuata sia tramite mrjob che PySpark. Di seguito i risultati ottenuti:

Tipo Veicolo	Media 2015 (€)	Media 2025 (€)	Variazione Assoluta (€)	Variazione (%)
Camion	19.93	23.98	+4.05	+20.32%
Autobus	14.56	17.29	+2.73	+18.75%
Furgone	11.47	13.9	+2.43	+21.19%
Auto	8.51	10.19	+1.68	+19.74%
Moto	5.73	6.9	+1.17	+20.42%

Osservazioni sui Dati:

Trend Economico: I dati riflettono correttamente l'inflazione programmata (aumento tra il 15% e il 25%), mostrando un aumento medio dei pedaggi coerente per tutte le categorie.

3. Analisi delle Performance (Quantitativa)

È stato misurato il tempo di esecuzione ("Wall time") per entrambe le implementazioni su un dataset di 100.000 righe in modalità locale.

Tempo mrjob: 0.47 secondi

Tempo PySpark: 1.33 secondi

Considerazioni: Sebbene PySpark sia tecnologicamente più avanzato, in questo specifico test su un dataset di dimensioni ridotte (Small Data), risulta più lento.

Motivazione: PySpark deve avviare la JVM e inizializzare il contesto Spark (SparkContext), operazioni che introducono un overhead iniziale significativo.

MrJob, lavorando con script Python puri in streaming, ha un tempo di avvio quasi nullo.

Scalabilità: Si ipotizza che all'aumentare del volume dei dati (es. milioni di record), la curva si invertirebbe: l'elaborazione parallela ottimizzata di Spark supererebbe la velocità lineare di Python.

4. Confronto Implementativo (Qualitativo)

Le due implementazioni evidenziano differenze sostanziali nello sviluppo:

A. MrJob (MapReduce Classico)

Paradigma: Imperativo. È necessario definire esplicitamente le classi Mapper, Combiner e Reducer.

Gestione Dati: Il parsing è manuale. Nel codice prodotto, è stato necessario gestire lo split delle stringhe CSV e la conversione dei tipi.

Ottimizzazione (Combiner): Per ridurre il traffico di rete (simulato), è stato implementato esplicitamente il metodo combiner. Questo aggrega i vettori parziali [conteggio, somma] direttamente nel nodo mapper prima dell'invio al reducer.

B. PySpark

Paradigma: Funzionale. A differenza dei DataFrame (dichiarativi), qui è stato utilizzato l'approccio RDD, che offre un controllo granulare sulle fasi di Map e Reduce, simile al paradigma puro, ma con la potenza del calcolo in memoria.

Ottimizzazione: Invece di affidarsi all'optimizer automatico (Catalyst), l'ottimizzazione è stata implementata a livello di codice. È stata utilizzata una logica "Single-Pass": un vettore accumulatore ha permesso di calcolare le medie di entrambi gli anni con un unico passaggio di shuffle (reduceByKey), dimezzando il costo di rete rispetto a un approccio ingenuo.

Combiner Implicito: Utilizzando la trasformazione reduceByKey, PySpark applica automaticamente una pre-aggregazione (Combiner) sui nodi mapper prima dello shuffle, replicando l'efficienza ottenuta manualmente in mrjob.

5. Tabella Riassuntiva

Aspetto	**mrjob**	**PySpark**
Tempo esecuzione	0.47s	1.33s
Paradigma	Imperativo	Funzionale
Combiner	Definito esplicitamente	Implicito in reduceByKey
Ottimizzazione	Combiner Manuale	Vettoriale Single-Pass
Scalabilità	Limitata (Python puro)	Eccellente (JVM distribuita)